NEXUS ONE SECURITY

SMART CONTRACT SECURITY AUDIT

Studio Miria Review

JULY 2026 · CONFIDENTIAL

OVERVIEW

Introduction

A time boxed security review of the **Studio Miria** protocol was performed by **Nexus One Security**, with a focus on the security aspects of the smart contract implementation. The audit targeted the project NFT minting and migration suite with the objective of identifying and mitigating potential vulnerabilities, thereby strengthening the security and robustness of the blockchain infrastructure.

About Studio Miria

Studio Miria is a Move based NFT platform. This review focuses on the minting and migration flow, including the mint and migration warehouses, image asset management, whitelist and migration ticket issuance, and the administrative controls that govern the minting process.

About Nexus One Security

Nexus One Security delivers independent smart contract audits and formal verification, reading code line by line to prove what is broken and remaining a long term security partner after the report is delivered. We secure protocols across Move on Sui and Aptos, Solidity on EVM chains, and Rust on Solana and beyond.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise bound effort where we try to find as many vulnerabilities as possible. We cannot guarantee 100% security after the review, nor that the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs, and on chain monitoring are strongly recommended.

OVERVIEW

Audit Summary

This report outlines the findings from a security review of the Studio Miria smart contract suite. The audit revealed the findings summarized below. Recommendations for remediation are provided to address them effectively.

0	1	0	5
CRITICAL	HIGH	MEDIUM	INFORMATIONAL

Severity Classification

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact the technical, economic, and reputation damage of a successful attack.

Likelihood the chance that a particular vulnerability is discovered and exploited.

Severity the overall criticality of the risk.

Scope

- Studio Miria minting and migration module
- Image asset management and deletion flow

DETAILED FINDINGS

High Severity

N1-01 Mismatched Image IDs Allow Unintended Image Deletion

HIGH

LOCATION	Studio Miria image asset module Function: `delete_image` Objects: `DeleteImagePromise`, `Image`
DESCRIPTION	The `DeleteImagePromise` and the `Image` object carry separate `image_id` references that must match to ensure the correct image is targeted for deletion. There is a risk that a mismatch between these IDs could lead to unintended image deletions, affecting data integrity and system reliability. The relationship between the `image_id` in `DeleteImagePromise` and the `id` in the `Image` object is crucial for the safe deletion of images. When the deletion path does not enforce that these IDs match, the wrong image can be deleted.
IMPACT	• **Data integrity.** Mismatched IDs could result in the wrong image being deleted, leading to loss of important data and affecting the consistency of the system state. • **System reliability.** Unintended deletions could undermine the reliability of the system, as users and administrators may lose trust in the ability of the system to manage image assets correctly. • **Security.** If the mechanism that matches image IDs is not robust, it could be exploited to maliciously delete images, posing a security risk.
RECOMMENDATION	Implement strict validation checks in the `delete_image` function to ensure that the `image_id` in the `DeleteImagePromise` matches the `id` of the `Image` object before proceeding with the deletion. ``` public fun delete_image(promise: DeleteImagePromise, image: Image) { // Ensure the promise targets this exact image before deleting assert!(promise.image_id == image.id, error::invalid_argument(EIMAGE_ID_MISMATCH)); // ... proceed with deletion ... } ``` Complement the validation with enhanced logging and monitoring, effective error handling, and strict access controls so the system can safeguard against unintended deletions and improve overall security and trustworthiness.
RESOLUTION	OPEN

Informational

N1-02 Centralization of Power in Administrative Functions

INFO

LOCATION	Studio Miria administrative module, multiple functions
DESCRIPTION	Several administrative functions centralize significant power in a single admin address. If that address is compromised, the concentrated control over minting, migration, warehouses, and refunds becomes a vulnerability. The concerns identified are: **`admin_add_to_mint_warehouse`** Centralizes the power to add NFTs to the mint warehouse, which is a risk if administrative access is compromised. Consider decentralized mechanisms or multi signature requirements for critical actions to distribute this power. **`admin_add_to_migration_warehouse`** Gives administrators significant control over the migration process. Ensure strict access controls and possibly introduce checks or balances such as multi sig approvals to reduce the risk of misuse. **`admin_destroy_migration_warehouse, admin_destroy_mint_warehouse`** The destruction of warehouses should be handled with extreme caution to prevent accidental loss of data or assets. Implement additional checks, confirmation steps, or a delay mechanism to allow for audit and reversal in case of an error or malicious action. **`admin_set_mint_phase, admin_set_mint_price, admin_set_mint_status`** These functions allow administrators to directly influence the economic and operational parameters of the minting process. To avoid centralization risks and potential manipulation, consider governance mechanisms where such critical parameters can be voted on by the community or a group of stakeholders. **`admin_issue_migration_ticket, admin_issue_whitelist_ticket`** Issuing tickets is a sensitive operation that can affect the fairness and transparency of the minting and migration processes. Implement transparent criteria and logs for ticket issuance, along with oversight mechanisms, to mitigate potential abuses of power. **`admin_refund_mint, admin_reveal_mint`** Refund and reveal operations are critical and sensitive, carrying the risk of financial loss or revealing incorrect information. These functions should have strict validation checks and perhaps a secondary confirmation to ensure correctness and prevent abuse.
RECOMMENDATION	Implement decentralized mechanisms, such as multi signature requirements and checks or balances, to ensure that no single entity has complete control over critical actions like adding NFTs to warehouses, managing the minting process, or executing refunds and reveals. Introduce governance mechanisms for setting important parameters and transparent criteria for ticket issuance to distribute power, increase transparency, and prevent potential abuses. This approach enhances security and promotes fairness and trust by ensuring that critical actions are subject to collective decision making and oversight.
RESOLUTION	ACKNOWLEDGED

N1-03 Unused Code Increases Maintenance Surface

INFO

LOCATION	Studio Miria minting and migration module
DESCRIPTION	The module contains constants, imports, and a helper function that are declared but never referenced anywhere in the codebase. Dead code adds noise, can mislead readers about intended behavior, and may hide the fact that a guard or limit was meant to be enforced but never wired up. `const MAX_MINT_PER_WALLET: u64 = 10;` `const DEFAULT_MINT_PRICE: u64 = 100000000;` `fun remaining_supply(warehouse: &MintWarehouse): u64 {` `  warehouse.total - warehouse.minted` `}`
RECOMMENDATION	Remove the unused constants and the `remaining_supply` helper to keep the module clean and maintainable. If a limit such as `MAX_MINT_PER_WALLET` is intended to be enforced, wire it into the relevant minting validation rather than leaving it as dead code.
RESOLUTION	ACKNOWLEDGED

N1-04 Fold a Vector Into a Single Value

INFO

LOCATION	Studio Miria minting and migration module Function: `total_minted_supply` Type: `MintWarehouse`
DESCRIPTION	The routine that totals minted tokens across the mint warehouses is written as a manual mutable index loop. Move supports the `fold!` macro, which expresses the same aggregation more concisely, removes mutable state, and reduces the chance of off by one or index errors. **Current pattern.** `let mut total = 0;` `let mut i = 0;` `while (i < warehouses.length()) {` `  total = total + warehouses[i].minted;` `  i = i + 1;` `};`
RECOMMENDATION	Use the `fold!` macro to sum the minted counts in a single expression. `let total = warehouses.fold!(0, |acc, w| {` `  acc + w.minted` `});`
RESOLUTION	ACKNOWLEDGED

N1-05 Filter Elements of a Vector

INFO

LOCATION	Studio Miria minting and migration module Function: `unrevealed_token_ids`
DESCRIPTION	Selecting the token IDs that are still awaiting reveal is implemented with a manual loop that pushes matching elements into a new vector. Since the `u64` token IDs have the `drop` ability, Move supports the `filter!` macro, which is clearer and less error prone. **Current pattern.** `let mut unrevealed = [];` `let mut i = 0;` `while (i < token_ids.length()) {` `  if (token_ids[i] > reveal_cursor) {` `  unrevealed.push_back(token_ids[i]);` `  };` `  i = i + 1;` `};`
RECOMMENDATION	Use the `filter!` macro to keep only the token IDs that are still unrevealed. `let unrevealed = token_ids.filter!(|id| id > reveal_cursor);`
RESOLUTION	ACKNOWLEDGED

N1-06 Ignored Values in Unpack Can Be Ignored Altogether

INFO

LOCATION	Studio Miria image asset module Function: `delete_image` Type: `Image`
DESCRIPTION	When destroying an `Image` object to reclaim its `id`, the unpack binds every remaining field to `_` even though only the `id` is used. This is verbose and sparse. The 2024 Move syntax allows the remaining fields to be ignored with `..`, keeping only the field that is actually used. **Current pattern.** `let Image { id, image_id: _, url: _, owner: _ } = image;` `id.delete();`
RECOMMENDATION	Use the 2024 `..` syntax to ignore the unused fields. `let Image { id, .. } = image;` `id.delete();`
RESOLUTION	ACKNOWLEDGED